CS4475 Fall 2026 Lab Report
Naomi Ortega Armendariz, Submitted on October 14, 2026
nortegaarmend@miners.utep.edu

PA2: Priority-based Scheduler for xv6

Please replace red text with your report text and any tables or figures, names of any accompanying files, etc. Remember to commit all the files for your lab submission, to put the URL for your private xv6 repo in the Teams assignment, to submit the Teams assignment, and to give the instructor and TA access to your repo.

Task 1. (15 pts) Implement the getprocs() system call and test it using the provided ps command.

Describe how you implemented the getprocs() system call and list what files you had to change. Show the results of testing getprocs() by running the ps command.

I implemented getprocs() as a new system call, the same way I added wait2 in PA1. getprocs() fills an array of struct pstat with the info of each process (pid, state, size, parent pid and name) and returns how many processes there are. Then the ps program prints that array.

kernel/proc.h: I took out the enum procstate line and put #include "pstat.h" in its place. Now the enum lives in pstat.h, so both the kernel and ps.c can use it.

kernel > h proc.h

```
40   struct trapframe {
74     /* 264 */ uint64 t4;
75     /* 272 */ uint64 t5;
76     /* 280 */ uint64 t6;
77   };
78
79   #include "pstat.h"   // enum procstate now lives in pstat.h
80
81   // Per-process state
82   struct proc {
83     struct spinlock lock;
84
```

kernel/pstat.h: I added struct pstat, which holds the info ps prints for each process (pid, state, size, parent pid and name). I also moved enum procstate here, because ps.c is a user program and can't include proc.h. I added an include guard (#ifndef PSTAT_H) so the file can't be defined twice.

usys.pl M | time.c | Makefile M | pstat.h M | user.h M | proc.h M

kernel > pstat.h

```h
#ifndef PSTAT_H
#define PSTAT_H

// Process states (moved here from proc.h so user programs like ps can use it)
enum procstate { UNUSED, USED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };

// pstat.h: resource usage info returned to user programs by wait2()
struct rusage {
  uint cputime;   // CPU ticks the process used
};

// Process info returned by the getprocs() system call
struct pstat {
  int pid;               // Process ID
  enum procstate state;  // Process state
  uint64 size;           // Size of process memory (bytes)
  int ppid;              // Parent process ID
  char name[16];         // Process command name
};

#endif
```

user/user.h: I added struct pstat; so the compiler knows the struct exists, and the prototype int getprocs(struct pstat*); so user programs like ps can call it.

usys.pl M | time.c | Makefile M | pstat.h M | user.h M | proc.h M

user > user.h

```h
#define SBRK_ERROR ((char *)-1)

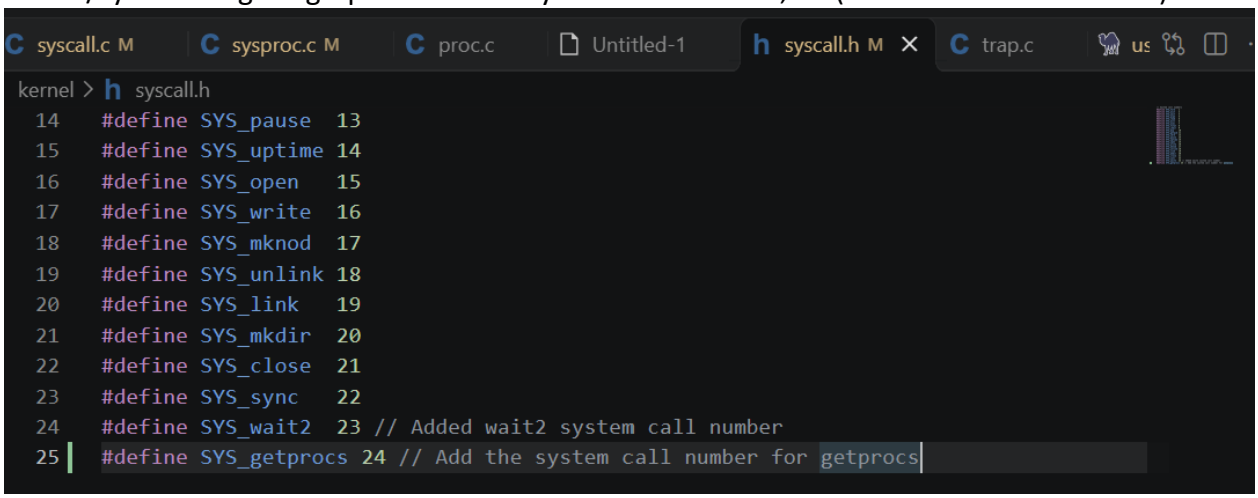
struct stat;
struct rusage; // Add struct rusage to hold the cputime of the child process
struct pstat; // Add to tell the compiler struct pstat exists so the getprocs prototy

// system calls
int fork(void);
int exit(int) __attribute__((noreturn));
int wait(int *);
int wait2(int*, struct rusage*); //Add wait2 to get the cputime of the child process
int getprocs(struct pstat*); //Add getprocs to get the info of all the processes for
int pipe(int *);
int write(int, const void *, int);
int read(int, void *, int);
int close(int);
int kill(int);
int exec(const char *, char **);
int open(const char *, int);
int mknod(const char *, short, short);
int unlink(const char *);
int fstat(int fd, struct stat *);
int link(const char *, const char *);
int mkdir(const char *);
int chdir(const char *);
int dup(int);
int getpid(void);
char *sys_sbrk(int, int);
int pause(int);
int uptime(void);
int sync(void);
```

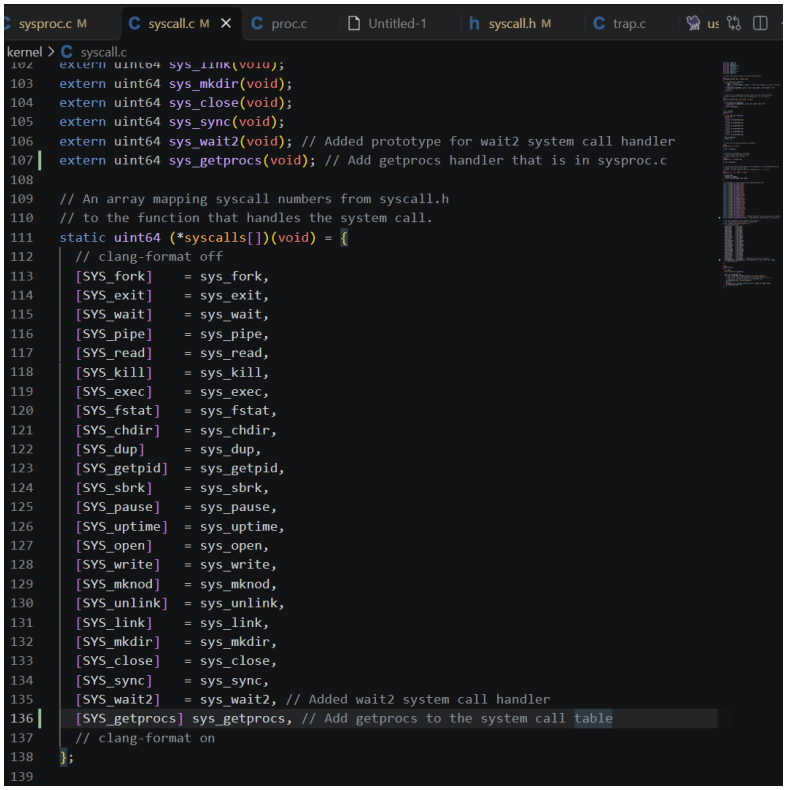
user/usys.pl: I added entry("getprocs"); This makes the small assembly stub that jumps from the user program into the kernel. The comment uses # because this is a Perl file.



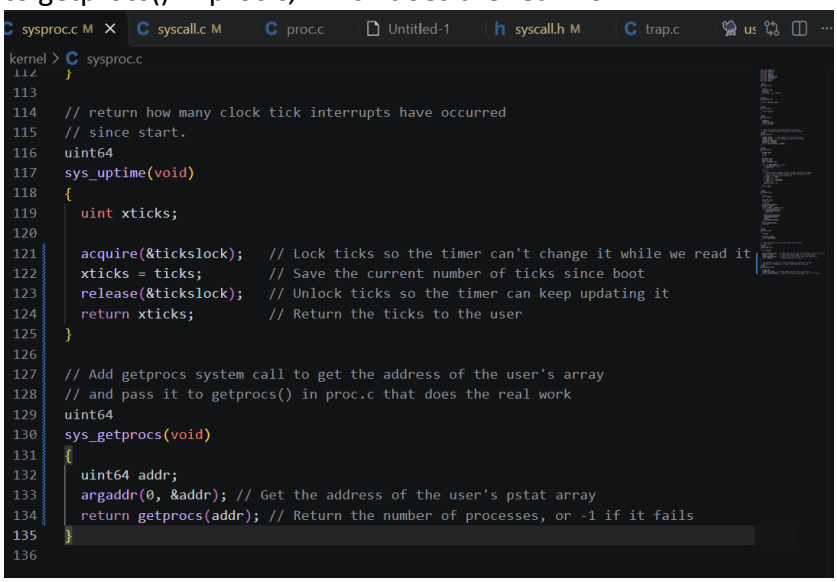
kernel/syscall.h: I gave getprocs its own system call number, 24 (the next one after wait2).



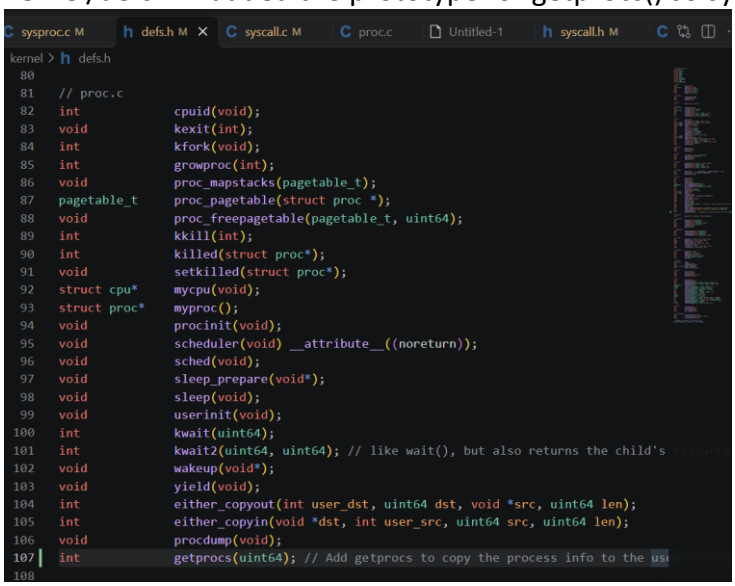
kernel/syscall.c: I added the extern for sys_getprocs and its entry in the syscalls[] table, so the kernel knows which function to run for number 24.



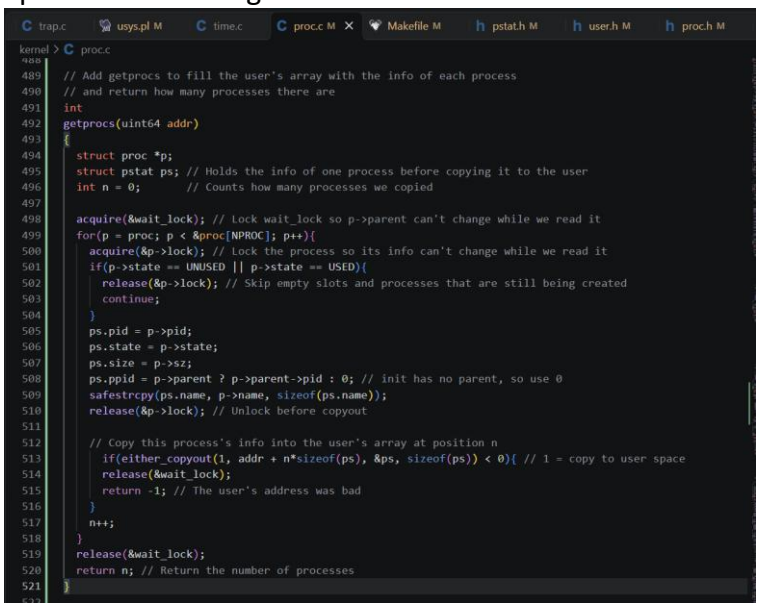
kernel/sysproc.c: sys_getprocs() gets the address of the user's array with argaddr() and passes it to getprocs() in proc.c, which does the real work.



kernel/defs.h: I added the prototype for getprocs() so sysproc.c can call it.

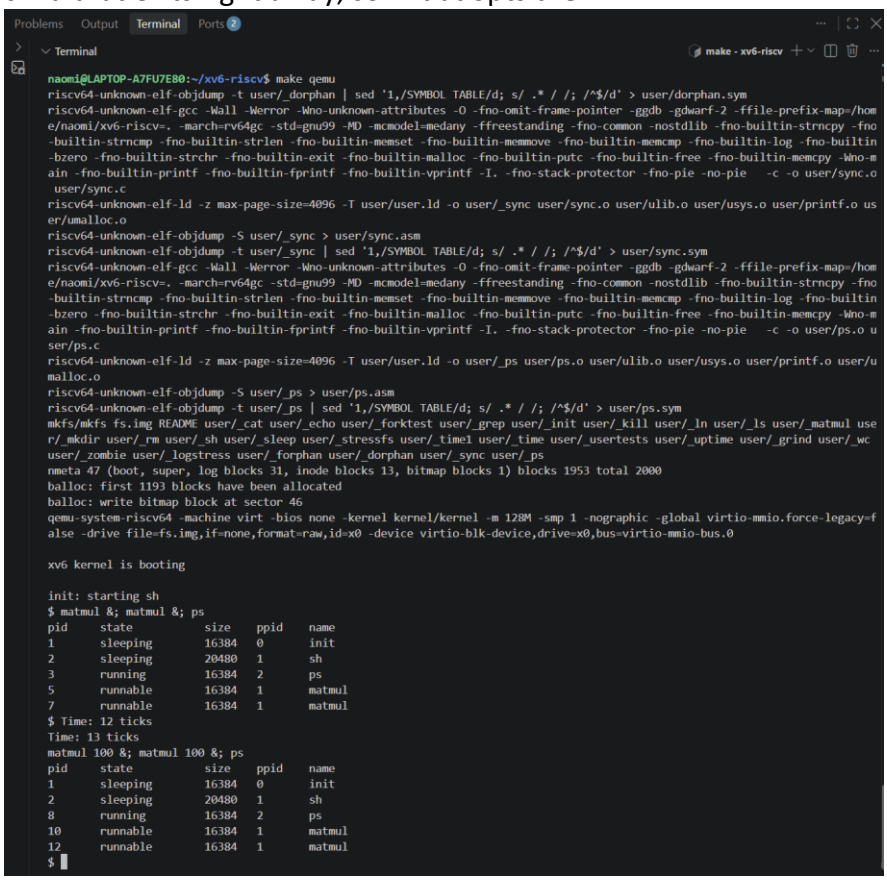


kernel/proc.c: getprocs() loops through the process table and skips UNUSED and USED slots. For each process it locks the process, copies its info into a struct pstat, unlocks it, and then copies the struct to the user's array with either_copyout(). It holds wait_lock during the loop so p->parent can't change while it reads it. It returns the number of processes it copied.



Testing: I ran ps with two matmul programs in the background (matmul &; matmul &; ps). ps shows init (pid 1), sh (pid 2, parent 1) and ps itself (running, parent 2). Both matmul processes

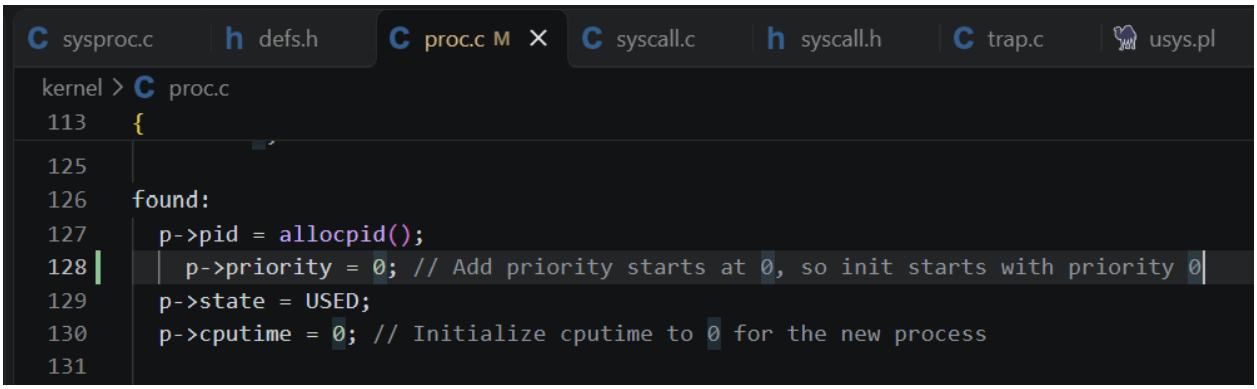
show as runnable while ps is running. Since we only use 1 CPU, only one process can be running at a time. The matmul processes have parent pid 1 because sh runs background commands in a child that exits right away, so init adopts them.



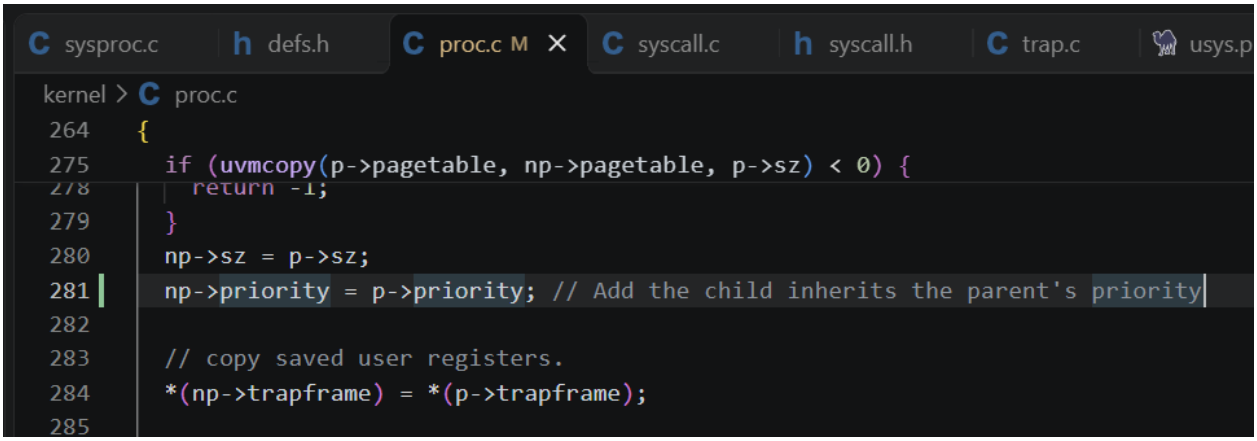
Task 2. (15 pts) Modify the provided ps command to print the priority of each process. Your modified ps.c should be in the user directory in the pa2 branch of your xv6 repo. Describe what changes you had to make to the getprocs() system call. Show the results of running your modified ps command.

I added a priority field to struct proc in proc.h. In allocproc() I set the priority to 0, so init starts with priority 0, and in kfork() the child copies its parent's priority. I added the getpriority() and setpriority() system calls with the same steps as getprocs (user.h, usys.pl, syscall.h, syscall.c and sysproc.c). setpriority only accepts priorities from 0 to 49 and returns -1 otherwise. It locks the process while it changes the priority so the scheduler can't read it at the same time.

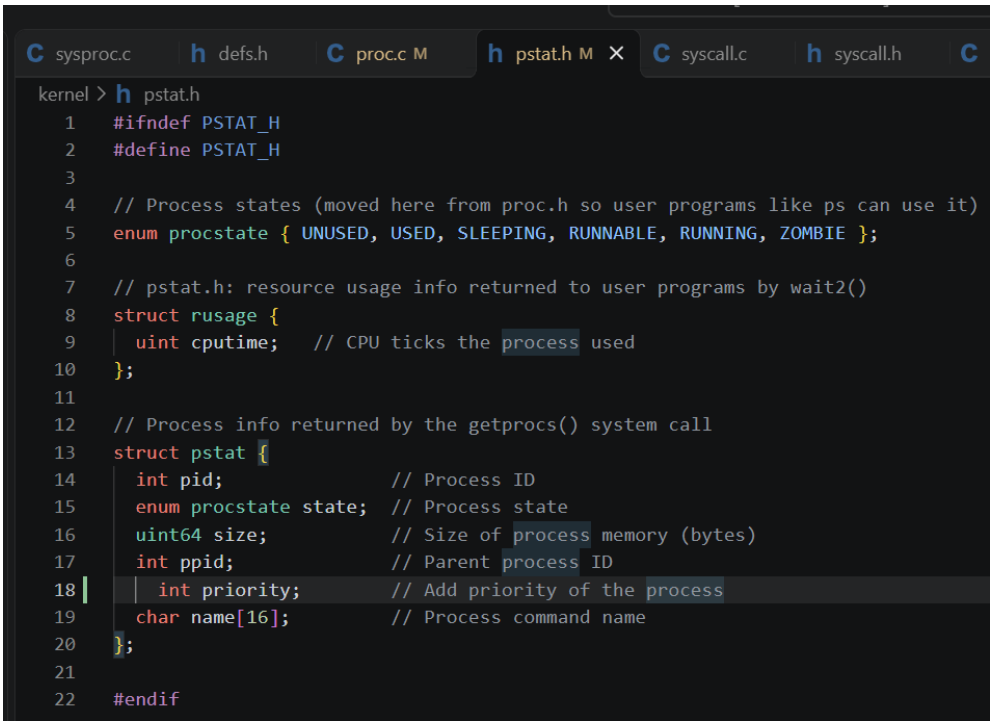
kernel/proc.c, allocproc(): right after the new process gets its pid, I set p->priority = 0. So every new process starts at 0, and init starts with priority 0.



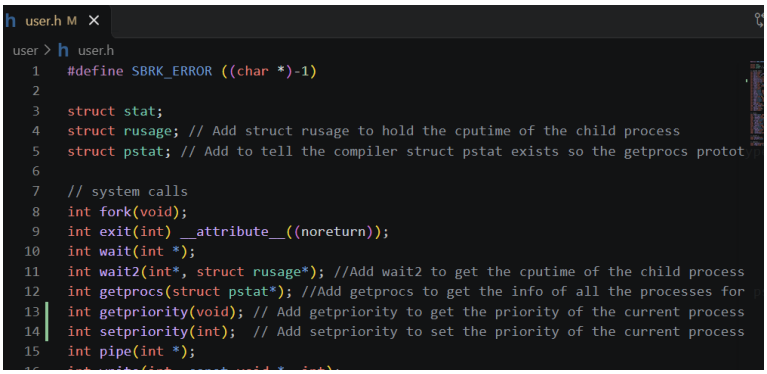
kernel/proc.c, kfork(): I added np->priority = p->priority; so the child inherits its parent's priority.



kernel/pstat.h: I added int priority to struct pstat so getprocs() can send the priority to ps.



user/user.h: I added the prototypes for getpriority() and setpriority().



user/usys.pl: I added the entries for getpriority and setpriority.

```
usys.pl M
user > usys.pl
39  entry("chdir");
40  entry("dup");
41  entry("getpid");
42  entry("sbrk");
43  entry("pause");
44  entry("uptime");
45  entry("sync");
46  entry("wait2"); # Added wait2, _syscall entry
47  entry("getprocs"); # Add getprocs to make the user stub that jumps into the kernel
48  entry("getpriority"); # Add getpriority user stub
49  entry("setpriority"); # Add setpriority user stub
```

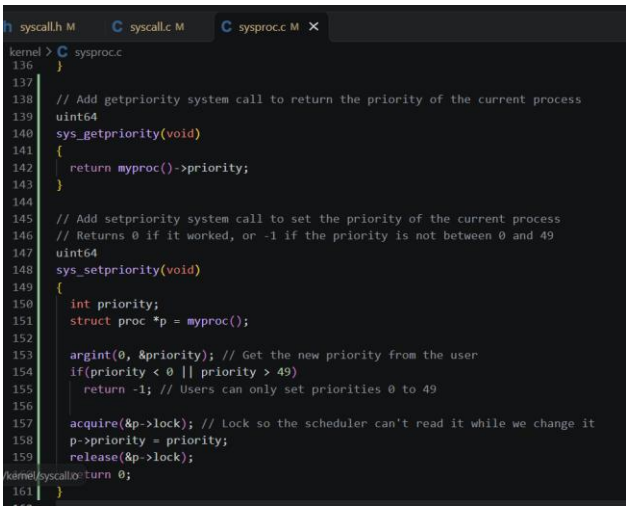
kernel/syscall.h: I gave them the system call numbers 25 and 26.

```
syscall.h M
kernel > syscall.h
14  #define SYS_pause  13
15  #define SYS_uptime 14
16  #define SYS_open   15
17  #define SYS_write  16
18  #define SYS_mknod  17
19  #define SYS_unlink 18
20  #define SYS_link   19
21  #define SYS_mkdir  20
22  #define SYS_close  21
23  #define SYS_sync   22
24  #define SYS_wait2  23 // Added wait2 system call number
25  #define SYS_getprocs 24 // Add the system call number for getprocs
26  #define SYS_getpriority 25 // Add the system call number for getpriority
27  #define SYS_setpriority 26 // Add the system call number for setpriority
```

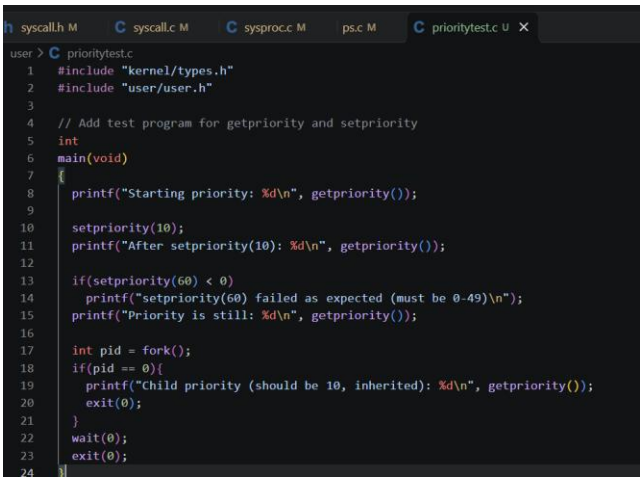
kernel/syscall.c: I added the externs and the table entries for sys_getpriority and sys_setpriority.

```
syscall.h M   syscall.c M
kernel > syscall.c
103 extern uint64 sys_mkdir(void);
104 extern uint64 sys_close(void);
105 extern uint64 sys_sync(void);
106 extern uint64 sys_wait2(void); // Added prototype for wait2 system call handler
107 extern uint64 sys_getprocs(void); // Add getprocs handler that is in sysproc.c
108 extern uint64 sys_getpriority(void); // Add getpriority handler that is in sysproc.c
109 extern uint64 sys_setpriority(void); // Add setpriority handler that is in sysproc.c
110
111 // An array mapping syscall numbers from syscall.h
112 // to the function that handles the system call.
113 static uint64 (*syscalls[])(void) = {
114   // clang-format off
115   [SYS_fork]    = sys_fork,
116   [SYS_exit]    = sys_exit,
117   [SYS_wait]    = sys_wait,
118   [SYS_pipe]    = sys_pipe,
119   [SYS_read]    = sys_read,
120   [SYS_kill]    = sys_kill,
121   [SYS_exec]    = sys_exec,
122   [SYS_fstat]   = sys_fstat,
123   [SYS_chdir]   = sys_chdir,
124   [SYS_dup]     = sys_dup,
125   [SYS_getpid]  = sys_getpid,
126   [SYS_sbrk]    = sys_sbrk,
127   [SYS_pause]   = sys_pause,
128   [SYS_uptime]  = sys_uptime,
129   [SYS_open]    = sys_open,
130   [SYS_write]   = sys_write,
131   [SYS_mknod]   = sys_mknod,
132   [SYS_unlink]  = sys_unlink,
133   [SYS_link]    = sys_link,
134   [SYS_mkdir]   = sys_mkdir,
135   [SYS_close]   = sys_close,
136   [SYS_sync]    = sys_sync,
137   [SYS_wait2]   = sys_wait2, // Added wait2 system call handler
138   [SYS_getprocs] sys_getprocs, // Add getprocs to the system call table
139   [SYS_getpriority] sys_getpriority, // Add getpriority to the system call table
140   [SYS_setpriority] sys_setpriority, // Add setpriority to the system call table
141   // clang-format on
```

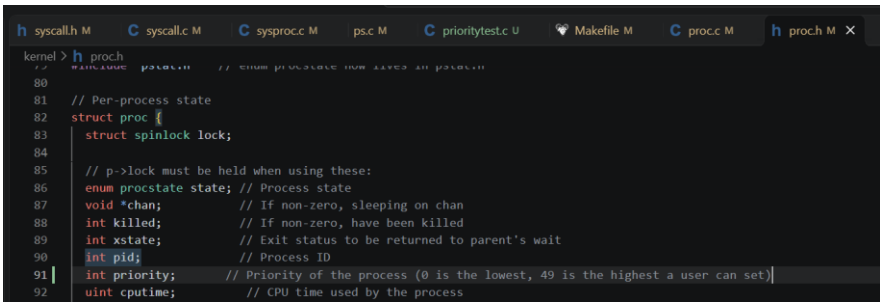
kernel/sysproc.c: sys_getpriority() returns the priority of the current process. sys_setpriority() gets the new priority with argint() and only accepts 0 to 49 (it returns -1 otherwise). It locks the process while it changes the priority so the scheduler can't read it at the same time.



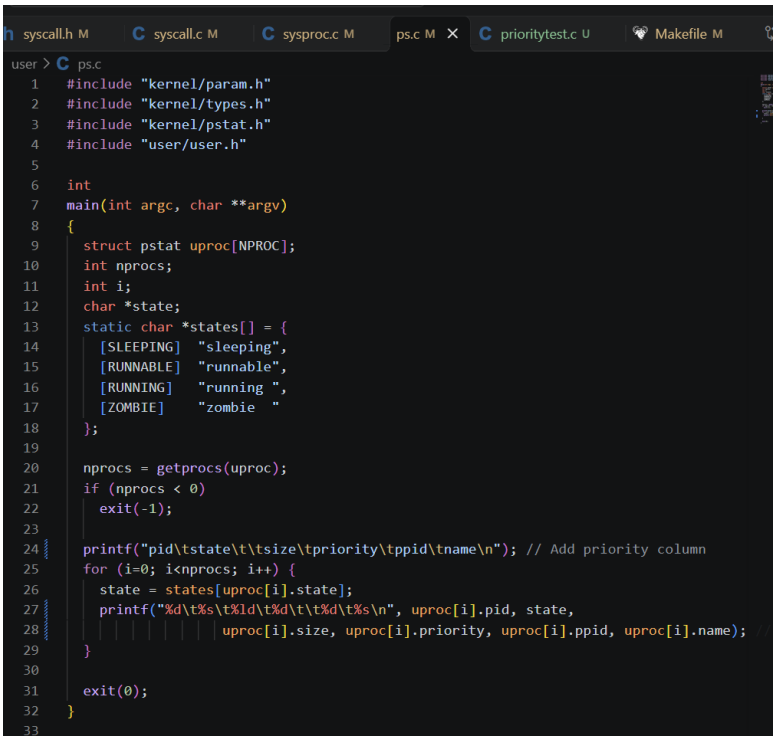
user/prioritytest.c: my test program. It prints the starting priority, sets it to 10, tries to set it to 60 (which should fail), and forks a child to check that the child inherits priority 10.



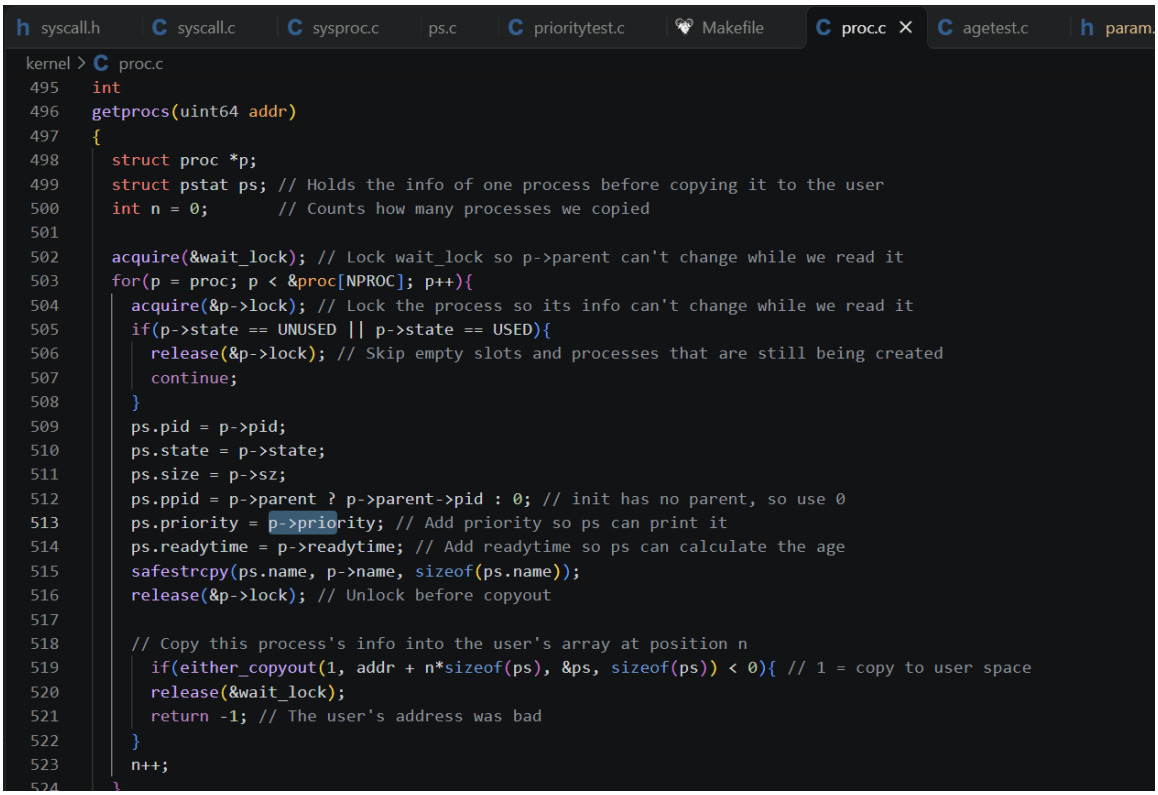
kernel/proc.h: I added int priority to struct proc, right after int pid.



user/ps.c: I added a priority column to the header and to the printf in the loop. I also changed %l to %ld, because size is a uint64 and the provided code didn't compile. In getprocs() I added ps.priority = p->priority; so the priority gets copied to the user.



kernel/proc.c, getprocs(): I added ps.priority = p->priority; (line 513) so the priority of each process gets copied to the user's array. This was the only change getprocs() needed for Task 2.



Testing: prioritytest shows the starting priority is 0, setpriority(10) works, setpriority(60) is rejected, and the child inherits priority 10. Then I ran pexec 5 matmul 100 &; pexec 10 ps. pexec and the matmul it started both have priority 5, and pexec and ps both have priority 10. This shows that children inherit their parent's priority. init and sh stay at 0.

```
naomi@LAPTOP-A7FU7E80:~/xv6-riscv$ make qemu
xec.sym
riscv64-unknown-elf-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-pointer -ggdb -
gdwarf-2 -ffile-prefix-map=/home/naomi/xv6-riscv=. -march=rv64gc -std=gnu99 -MD -mcmodel=medany
-ffreestanding -fno-common -nostdlib -fno-builtin-strncpy -fno-builtin-strncmp -fno-builtin-strl
en -fno-builtin-memset -fno-builtin-memmove -fno-builtin-memcmp -fno-builtin-log -fno-builtin-bz
ero -fno-builtin-strchr -fno-builtin-exit -fno-builtin-malloc -fno-builtin-putc -fno-builtin-fre
e -fno-builtin-memcpy -Wno-main -fno-builtin-printf -fno-builtin-fprintf -fno-builtin-vprintf -I
. -fno-stack-protector -fno-pie -no-pie   -c -o user/prioritytest.o user/prioritytest.c
riscv64-unknown-elf-ld -z max-page-size=4096 -T user/user.ld -o user/_prioritytest user/priority
test.o user/ulib.o user/usys.o user/printf.o user/umalloc.o
riscv64-unknown-elf-objdump -S user/_prioritytest > user/prioritytest.asm
riscv64-unknown-elf-objdump -t user/_prioritytest | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' >
user/prioritytest.sym
mkfs/mkfs fs.img README user/_cat user/_echo user/_forktest user/_grep user/_init user/_kill use
r/_ln user/_ls user/_matmul user/_mkdir user/_rm user/_sh user/_sleep user/_stressfs user/_time1
 user/_time user/_usertests user/_uptime user/_grind user/_wc user/_zombie user/_logstress user/
_forphan user/_dorphan user/_sync user/_ps user/_pexec user/_prioritytest
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
balloc: first 1269 blocks have been allocated
balloc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -nographic -gl
obal virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-b
lk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

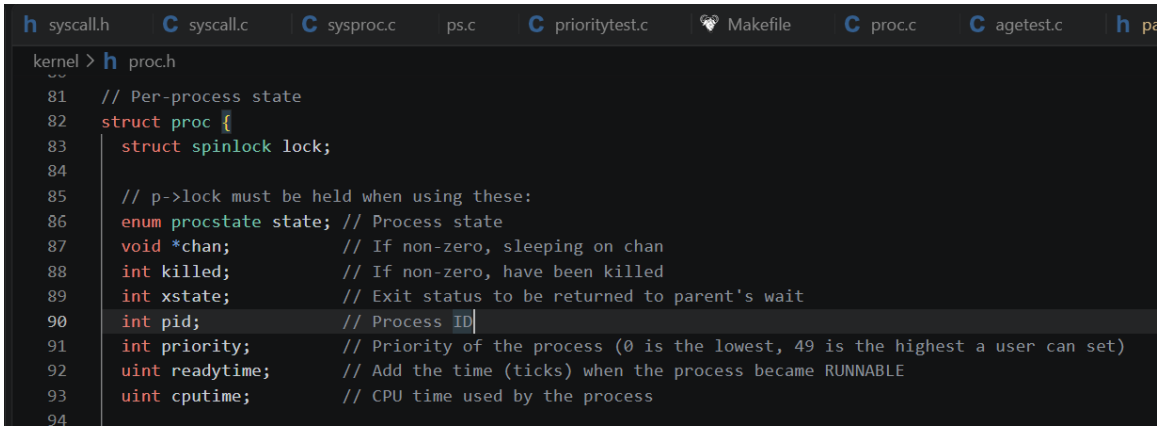
init: starting sh
$ prioritytest
Starting priority: 0
After setpriority(10): 10
setpriority(60) failed as expected (must be 0-49)
Priority is still: 10
Child priority (should be 10, inherited): 10
$ pexec 5 matmul 100 &; pexec 10 ps
pid     state           size    priority        ppid    name
1       sleeping        16384   0               0       init
2       sleeping        20480   0               1       sh
5       sleeping        16384   10              2       pexec
8       runnable        16384   5               7       matmul
7       sleeping        16384   5               1       pexec
9       running         16384   10              5       ps
$
```

Task 3. (15 pts) Add a readytime field to struct proc, initialize it correctly, and modify ps to print a process's age.

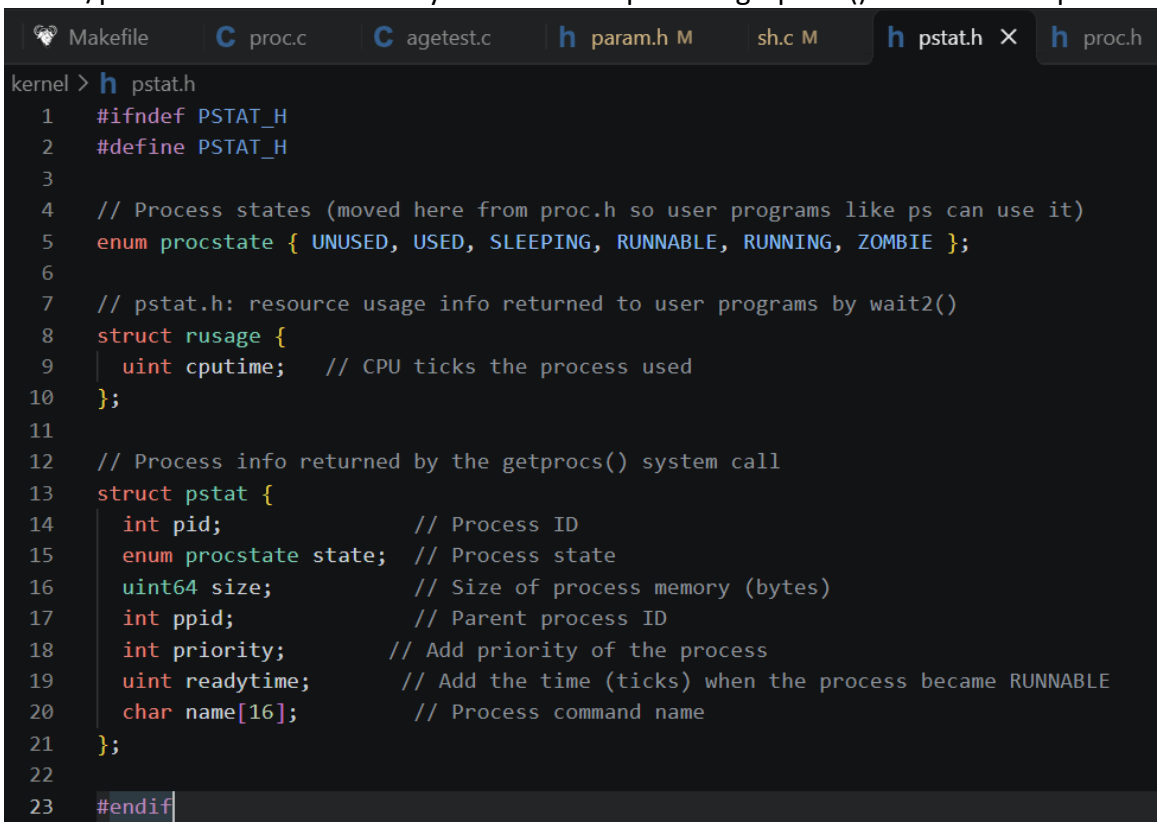
Explain how you calculate a process's age. Show the results of running your modified ps command.

I added a readytime field to struct proc in proc.h. Every time a process's state changes

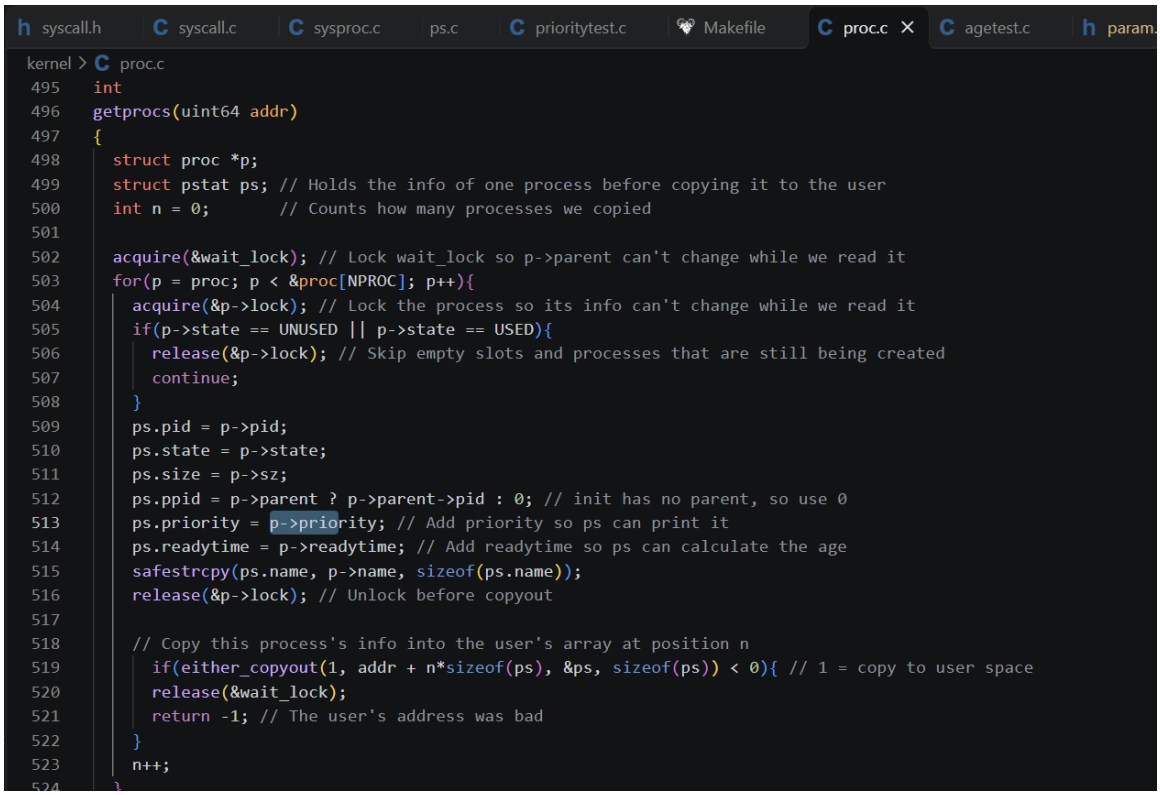
to RUNNABLE, I save the current time with readytime = ticks. This happens in 5 places in proc.c: userinit(), kfork(), yield(), wakeup() and kkill(). I also added readytime to struct pstat and to getprocs(), so ps can get it.
kernel/proc.h: I added uint readytime to struct proc, right after priority. It saves the time (in ticks) when the process became RUNNABLE.



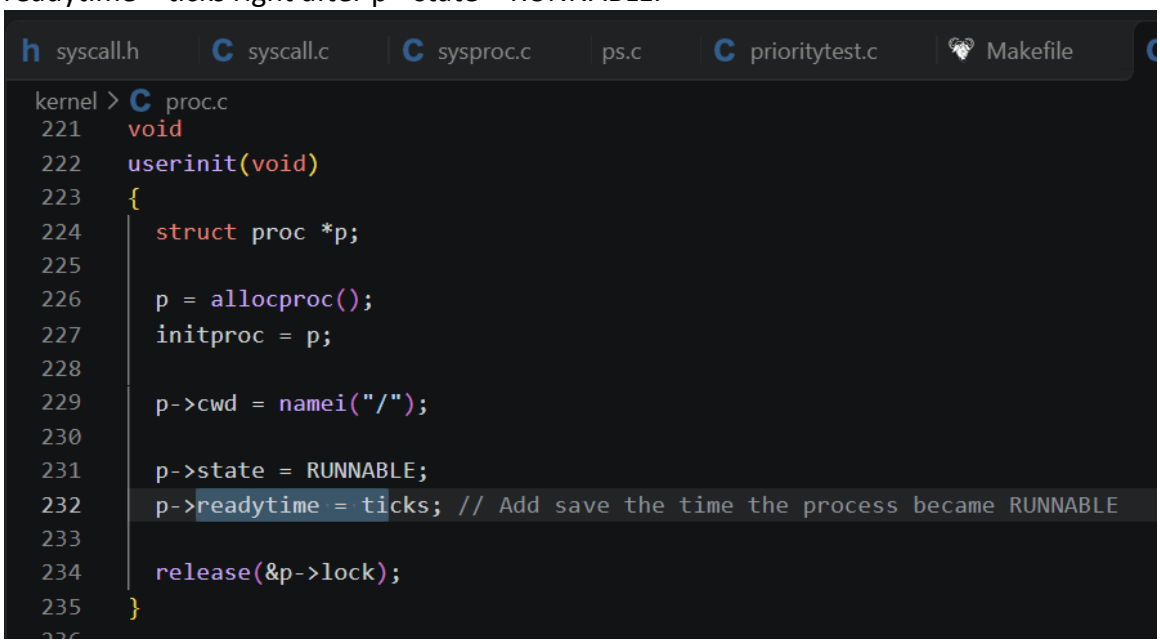
kernel/pstat.h: I added uint readytime to struct pstat so getprocs() can send it to ps.



kernel/proc.c, getprocs(): I added ps.readytime = p->readytime; so ps gets the readytime of each process (line 514).

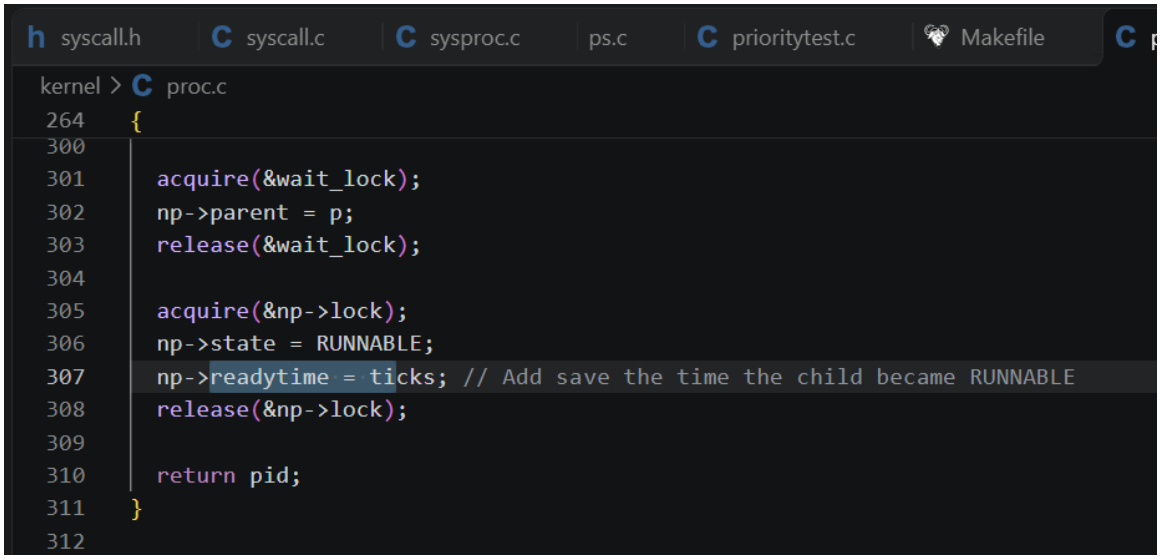


kernel/proc.c, userinit(): the first process (init) becomes RUNNABLE here, so I save readytime = ticks right after p->state = RUNNABLE.

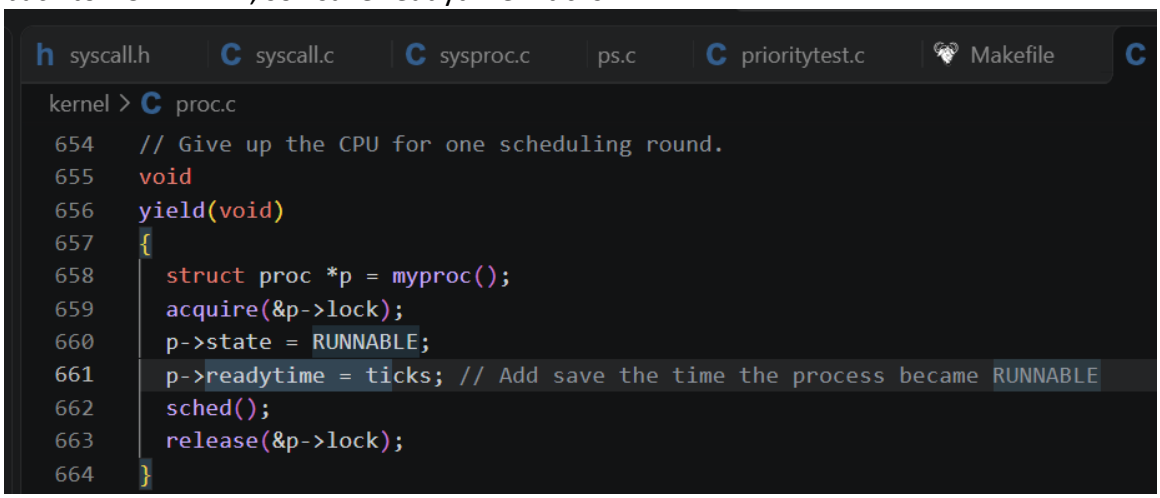


kernel/proc.c, kfork(): a new child becomes RUNNABLE at the end of fork, so I save np-

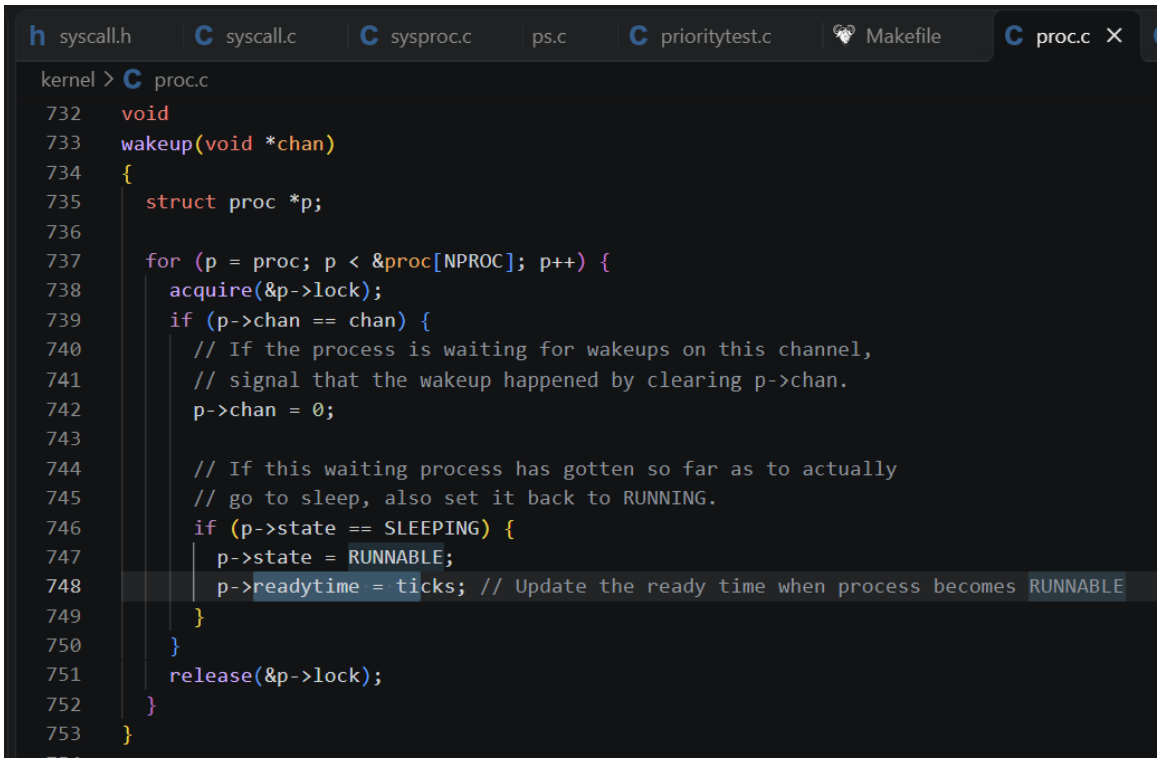
>readytime = ticks there.



kernel/proc.c, yield(): when the timer makes a running process give up the CPU, it goes back to RUNNABLE, so I save readytime = ticks.



kernel/proc.c, wakeup(): a sleeping process that wakes up becomes RUNNABLE, so I save readytime = ticks.



kernel/proc.c, kkill(): a killed process that was sleeping becomes RUNNABLE so it can exit, so I save readytime = ticks.

```
syscall.h | syscall.c | sysproc.c | ps.c | prioritytest.c | Makefile | proc
kernel > proc.c

754
755  // Kill the process with the given pid.
756  // The victim won't exit until it tries to return
757  // to user space (see usertrap() in trap.c).
758  int
759  kkill(int pid)
760  {
761    struct proc *p;
762
763    for (p = proc; p < &proc[NPROC]; p++) {
764      acquire(&p->lock);
765      if (p->pid == pid) {
766        p->killed = 1;
767        if (p->state == SLEEPING) {
768          // Wake process from sleep().
769          p->state = RUNNABLE;
770          p->readytime = ticks; // Add save the time the process became RUNNABLE
771        }
772        release(&p->lock);
773        return 0;
774      }
775      release(&p->lock);
776    }
777    return -1;
778  }
```

How I calculate the age: in ps.c I get the current time with uptime(). If a process is RUNNABLE, its age = current time - readytime, which is how many ticks it has been waiting in the ready queue. For every other state the age is 0.

```
syscall.h | syscall.c | sysproc.c | ps.c | prioritytest.c | Makefile | proc.c | agetest.c | param.h M | sh.c M | pstat.h | proc.h
user > ps.c

8   {
21    if (nprocs < 0)
24    uint now = uptime(); // Add get the current time to calculate the age
25
26    printf("pid\tstate\t\tsize\tage\tpriority\tppid\tname\n"); // Add age and priority columns
27    for (i=0; i<nprocs; i++) {
28      state = states[uproc[i].state];
29      uint age = 0;
30      if (uproc[i].state == RUNNABLE)
31        age = now - uproc[i].readytime; // Add age = current time - readytime, only for RUNNABLE processes
32      printf("%d\t%s\t%ld\t%d\t%d\t\t%d\t%s\n", uproc[i].pid, state,
33                uproc[i].size, age, uproc[i].priority, uproc[i].ppid, uproc[i].name); // Fix %l to %ld since size is uint64, and add age and priority
34    }
35
36    exit(0);
37  }
38
39
```

Testing: I ran matmul 200 &; matmul 200 &; ps. The runnable matmul shows age 1 while every other process shows 0. The ages are small because with round-robin the processes take turns every tick, so none of them waits long.

```
naomi@LAPTOP-A7FU7E80:~/xv6-riscv$ make qemu
en -fno-builtin-memset -fno-builtin-memmove -fno-builtin-memcmp -fno-builtin-log -fno-builtin-bz
ero -fno-builtin-strchr -fno-builtin-exit -fno-builtin-malloc -fno-builtin-putc -fno-builtin-fre
e -fno-builtin-memcpy -Wno-main -fno-builtin-printf -fno-builtin-fprintf -fno-builtin-vprintf -I
. -fno-stack-protector -fno-pie -no-pie   -c -o user/pexec.o user/pexec.c
riscv64-unknown-elf-ld -z max-page-size=4096 -T user/user.ld -o user/_pexec user/pexec.o user/ul
ib.o user/usys.o user/printf.o user/umalloc.o
riscv64-unknown-elf-objdump -S user/_pexec > user/pexec.asm
riscv64-unknown-elf-objdump -t user/_pexec | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > user/pe
xec.sym
riscv64-unknown-elf-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-pointer -ggdb -
gdwarf-2 -ffile-prefix-map=/home/naomi/xv6-riscv=. -march=rv64gc -std=gnu99 -MD -mcmodel=medany
-ffreestanding -fno-common -nostdlib -fno-builtin-strncpy -fno-builtin-strncmp -fno-builtin-strl
en -fno-builtin-memset -fno-builtin-memmove -fno-builtin-memcmp -fno-builtin-log -fno-builtin-bz
ero -fno-builtin-strchr -fno-builtin-exit -fno-builtin-malloc -fno-builtin-putc -fno-builtin-fre
e -fno-builtin-memcpy -Wno-main -fno-builtin-printf -fno-builtin-fprintf -fno-builtin-vprintf -I
. -fno-stack-protector -fno-pie -no-pie   -c -o user/prioritytest.o user/prioritytest.c
riscv64-unknown-elf-ld -z max-page-size=4096 -T user/user.ld -o user/_prioritytest user/priority
test.o user/ulib.o user/usys.o user/printf.o user/umalloc.o
riscv64-unknown-elf-objdump -S user/_prioritytest > user/prioritytest.asm
riscv64-unknown-elf-objdump -t user/_prioritytest | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' >
user/prioritytest.sym
mkfs/mkfs fs.img README user/_cat user/_echo user/_forktest user/_grep user/_init user/_kill use
r/_ln user/_ls user/_matmul user/_mkdir user/_rm user/_sh user/_sleep user/_stressfs user/_time1
 user/_time user/_usertests user/_uptime user/_grind user/_wc user/_zombie user/_logstress user/
_forphan user/_dorphan user/_sync user/_ps user/_pexec user/_prioritytest
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
balloc: first 1270 blocks have been allocated
balloc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -nographic -gl
obal virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-b
lk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

init: starting sh
$ matmul 200 &; matmul 200 &; ps
pid     state           size    age     priority        ppid    name
1       sleeping        16384   0       0               0       init
2       sleeping        20480   0       0               1       sh
3       running         16384   0       0               2       ps
5       runnable        16384   1       0               1       matmul
7       runnable        16384   0       0               1       matmul
$ ps
pid     state           size    age     priority        ppid    name
1       sleeping        16384   0       0               0       init
2       sleeping        20480   0       0               1       sh
8       running         16384   0       0               2       ps
5       runnable        16384   1       0               1       matmul
7       runnable        16384   0       0               1       matmul
$
```

Task 4. (15 pts) Implement a priority-based scheduler.

List the files and functions you changed and explain the purpose of each change. Be sure to do git add and git commit as needed and push your changes to your online repo. Show the results of running your test programs using the priority scheduler. Your tests should show higher priority programs getting to run first. You may use the provided pexec program for your tests.

kernel/proc.c, scheduler(): I added #if SCHEDULER == SCHED_PRIORITY. The new code looks through the process table for the RUNNABLE process with the highest priority and runs it. A bigger number means a higher priority. I kept the original round-robin code in the #else part, so it can still be used by changing param.h. If there is a tie, the processes take turns, because the search starts after the last process that ran.

Summarize what you learned by carrying out this task.

What I learned: a priority scheduler always runs the most important process first, but low priority processes can starve and never run.

Describe any difficulties you ran into with this task and if/how you overcame them.

Difficulties: In my first version a tie always went to the first process in the table. The shell's helper process had the same priority (0) as a matmul that was ahead of it, so it waited 385 ticks and never ran. I fixed it by making ties take turns. Also, the shell has priority 0, so it couldn't run while a higher priority matmul was running. I fixed my test by starting a shell with priority 20 using pexec 20 sh.

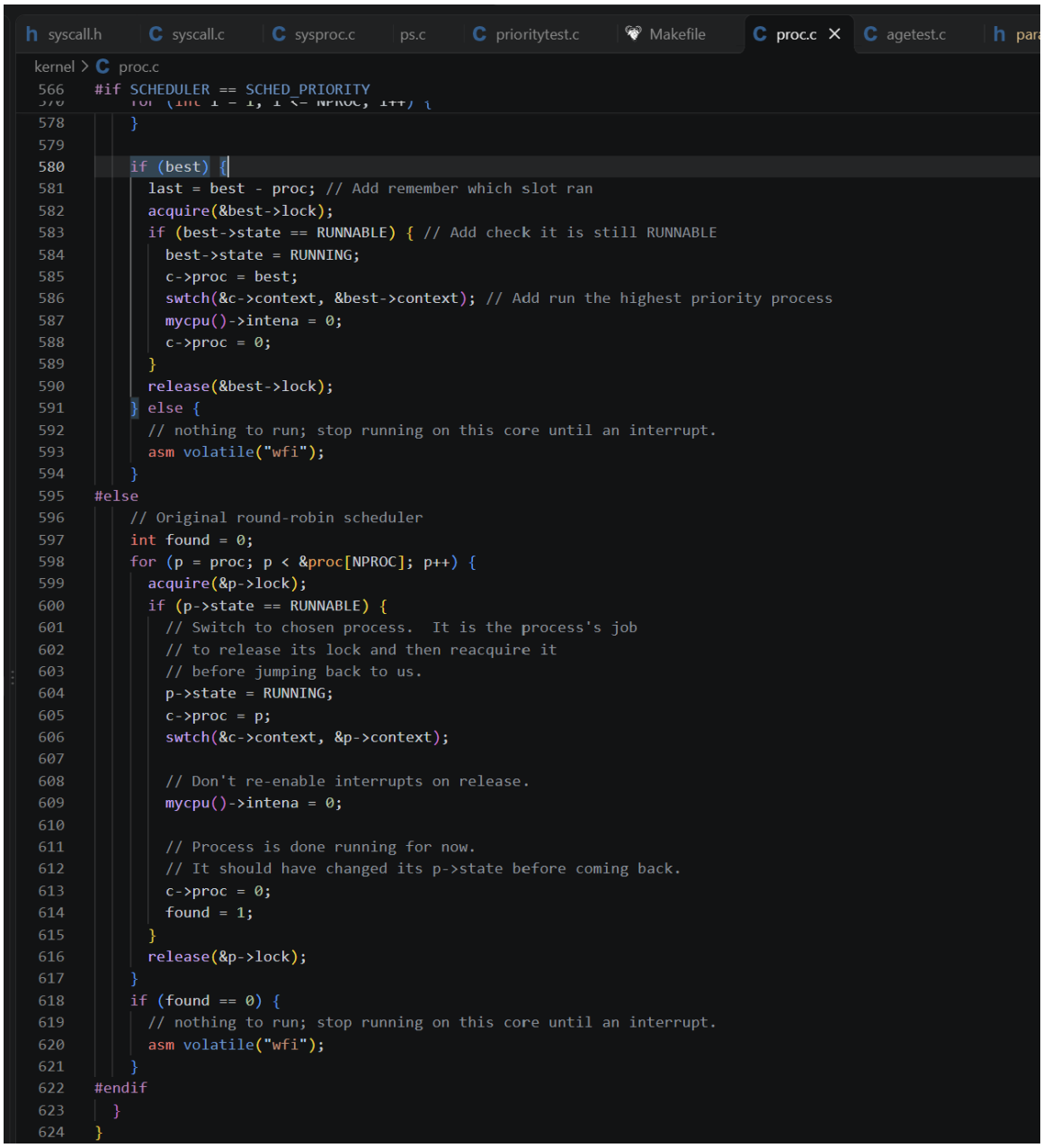
kernel/param.h: I added SCHED_RR, SCHED_PRIORITY and SCHEDULER to choose the scheduler at compile time. SCHEDULER is set to SCHED_PRIORITY.

```
all.c | sysproc.c | ps.c | prioritytest.c | Makefile | proc.c | param.h M
kernel > param.h
1  #define NPROC        64              // maximum number of processes
2  #define NCPU         8               // maximum number of CPUs
3  #define NOFILE       16              // open files per process
4  #define NFILE        100             // open files per system
5  #define NINODE       50              // maximum number of active i-nodes
6  #define NDEV         10              // maximum major device number
7  #define ROOTDEV      1               // device number of file system root disk
8  #define MAXARG       32              // max exec arguments
9  #define MAXOPBLOCKS  10              // max # of blocks any FS op writes
10 #define LOGBLOCKS    (MAXOPBLOCKS * 3) // max data blocks in on-disk log
11 #define NBUF         (MAXOPBLOCKS * 3) // size of disk block cache
12 #define FSSIZE       2000            // size of file system in blocks
13 #define MAXPATH      128             // maximum file path name
14 #define USERSTACK    1               // user stack pages
15 // Add scheduler options, pick one with SCHEDULER (compile-time)
16 #define SCHED_RR       0   // round-robin (original xv6)
17 #define SCHED_PRIORITY 1   // priority-based
18 #define SCHEDULER SCHED_PRIORITY
19
```

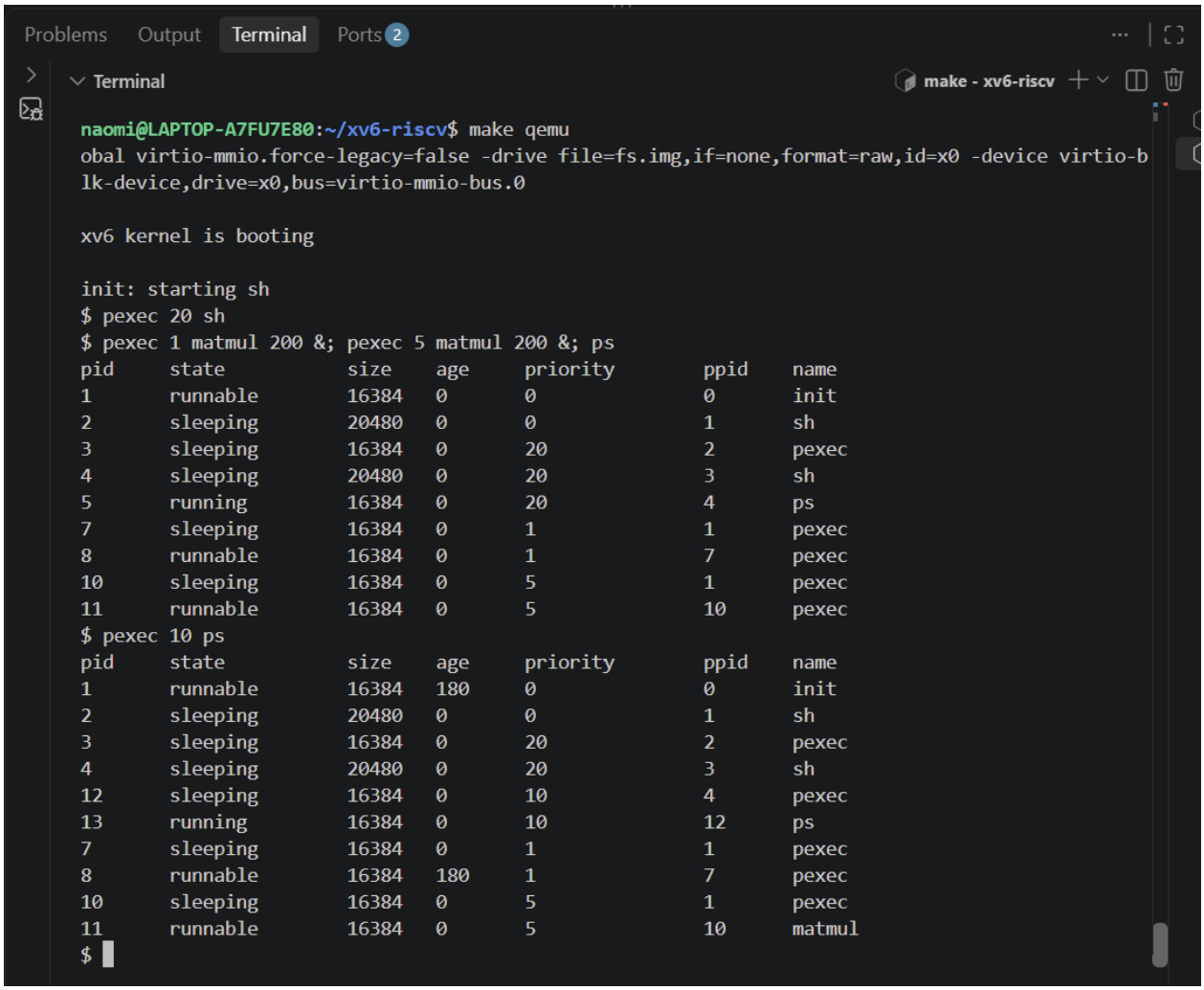
kernel/proc.c, scheduler(): the first part searches the whole process table, starting after the last process that ran, and keeps the RUNNABLE process with the highest priority in best. (These screenshots were taken after Task 5, so the code already compares effpriority(p), which is the priority with aging.)

```
syscall.h | syscall.c | sysproc.c | ps.c | prioritytest.c | Makefile | proc.c | agetest.c | param.h M
kernel > proc.c
551  {
557    for (;;) {
565
566  #if SCHEDULER == SCHED_PRIORITY
567      // Add priority scheduler: find the RUNNABLE process with the highest priority
568      struct proc *best = 0;
569      int bestprio = -1;
570      for (int i = 1; i <= NPROC; i++) {
571        p = &proc[(last + i) % NPROC]; // Add start searching after the last process that ran, so ties take turns
572        acquire(&p->lock);
573        if (p->state == RUNNABLE && effpriority(p) > bestprio) { // Add use the effective priority (with aging)
574          best = p; // Add this is the highest priority so far
575          bestprio = effpriority(p);
576        }
577        release(&p->lock);
578      }
579
```

If it found a process, it remembers its slot in last (so ties take turns next time), checks that it is still RUNNABLE, sets it to RUNNING and switches to it with swtch(). If nothing is RUNNABLE, the CPU waits for an interrupt with wfi. The original round-robin scheduler is kept in the #else part, so it can be chosen again by changing SCHEDULER in param.h.



Testing: I ran this test before adding aging (AGING 0), so the scheduler used only the normal priority. I ran pexec 20 sh to get a shell with priority 20, and then pexec 1 matmul 200 &; pexec 5 matmul 200 &; ps. In the second ps, the priority 5 matmul gets the CPU, while the priority 1 process has been waiting 180 ticks and never ran once (it still has the name pexec because it never got to start matmul). ps runs right away because it has a higher priority than everything else. This shows that higher priority programs run first, and that low priority processes starve without aging.



Task 5. (15 pts) Add aging to your priority-based scheduler.

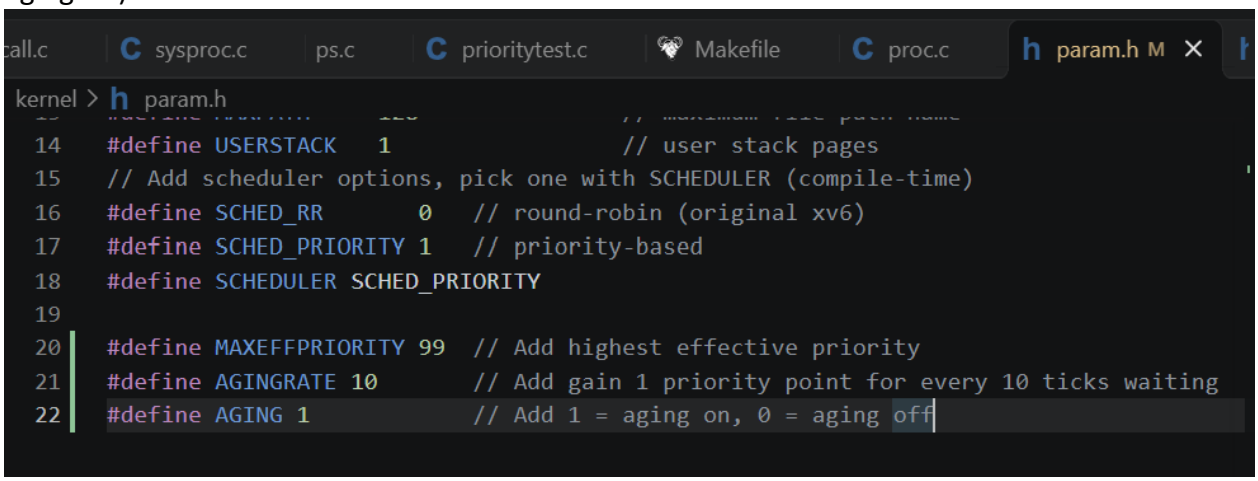
Explain your aging policy. Show results from running your priority-based scheduler with aging. Your tests should show the benefits of using aging.

Aging policy: effective priority = priority + (ticks waiting in the ready queue / 10), with a max of 99. So a process gains 1 priority point for every 10 ticks it waits. I added MAXEFFPRIORITY, AGINGRATE and AGING to param.h (AGING 1 turns aging on, 0 turns it off). I added an effpriority() function in proc.c, and the scheduler now compares effective priorities instead of normal priorities.

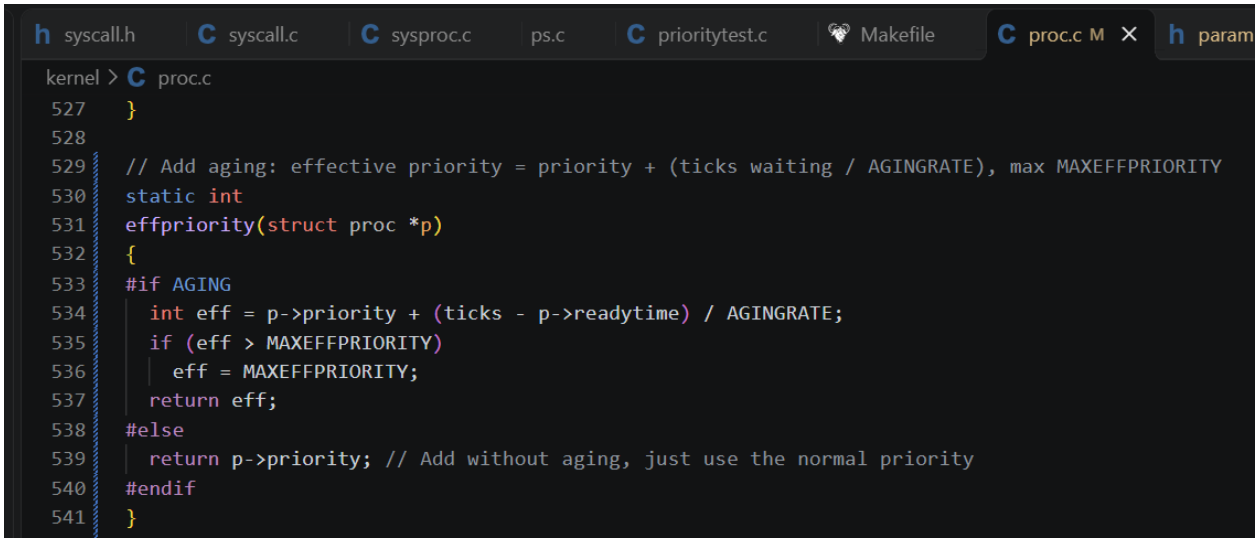
Summarize what you learned by carrying out this task.

What I learned: aging fixes starvation by slowly raising the priority of processes that wait a long time. There is a trade-off: high priority processes get delayed a little so low priority ones get a turn.

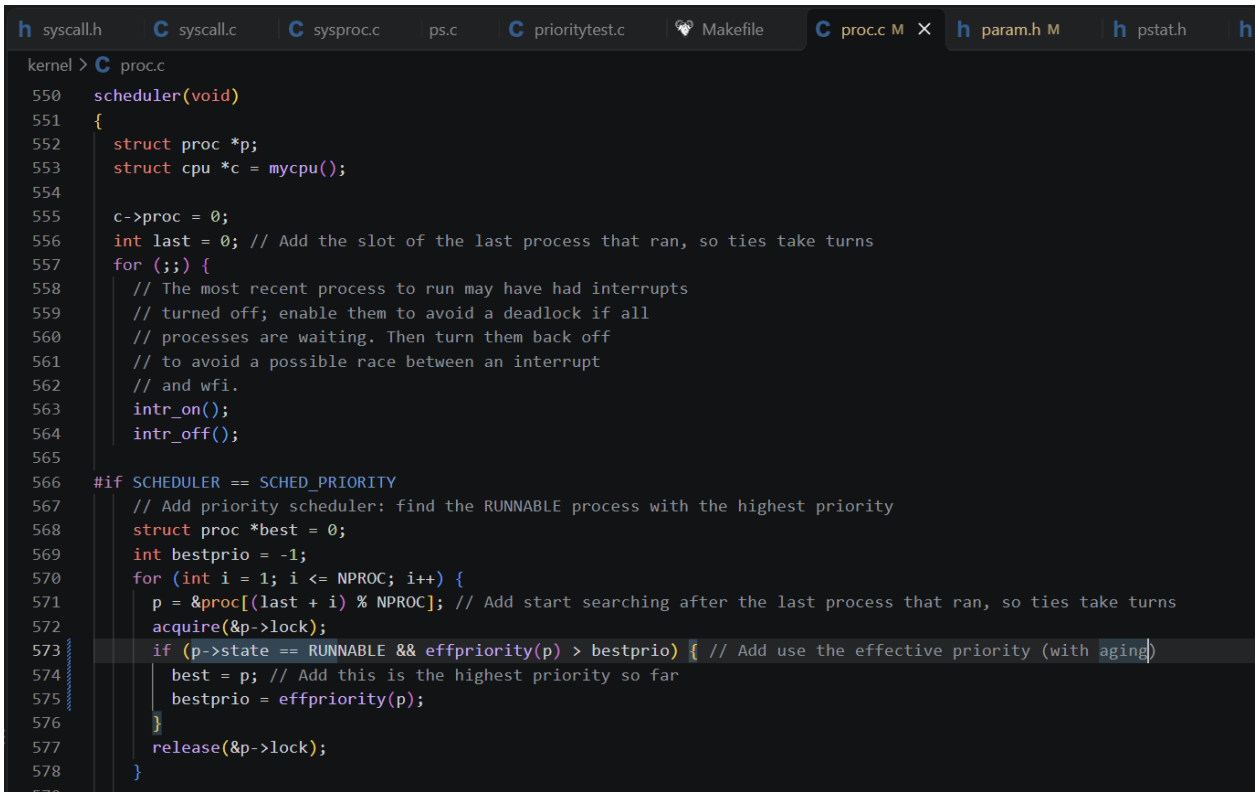
Describe any difficulties you ran into with this task and if/how you overcame them.
Difficulties: my first test only had about 3 ticks of work per process, so aging never had time to make a difference and both results looked the same. I fixed it by giving each process more work (agetest 2000).
kernel/param.h: I added MAXEFFPRIORITY (99), AGINGRATE (10) and AGING (1 = aging on, 0 = aging off).



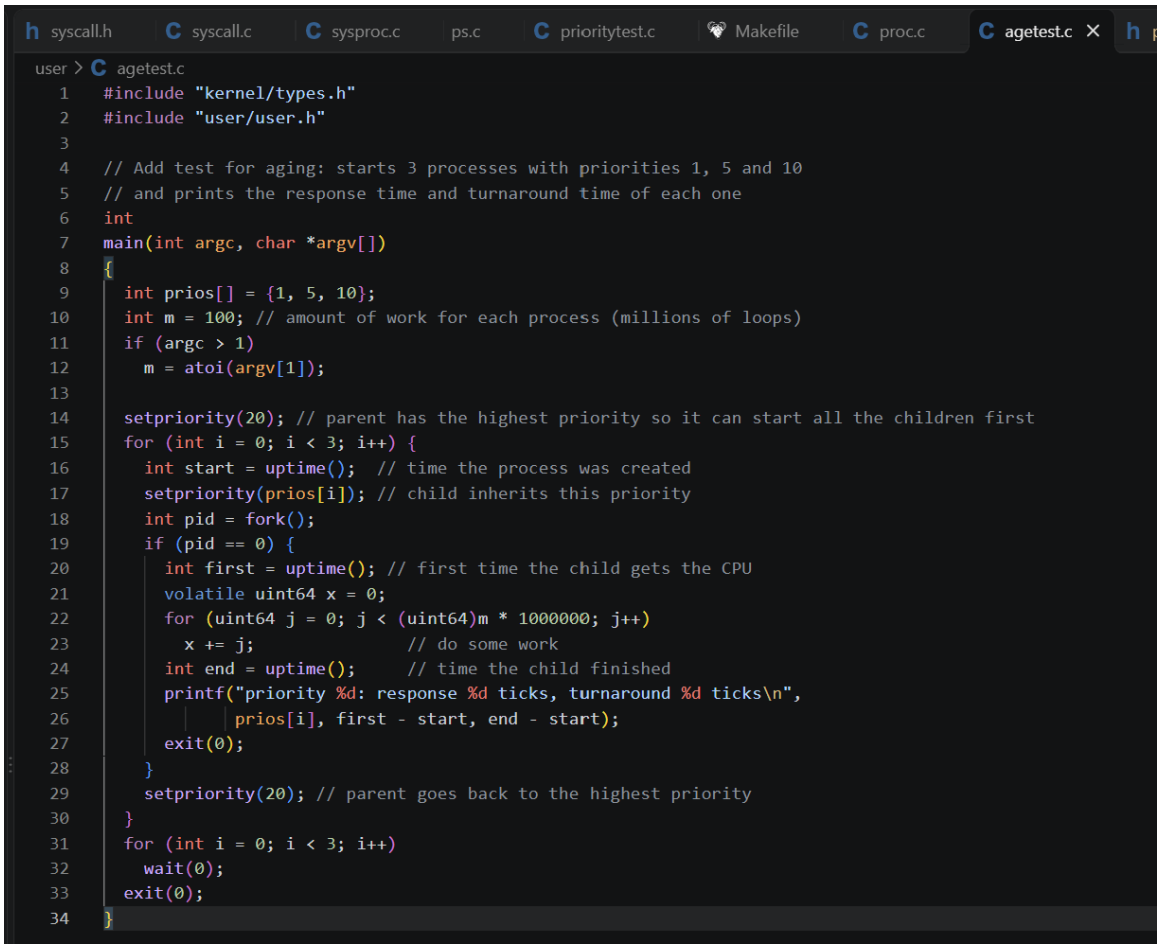
kernel/proc.c: effpriority() calculates the effective priority: priority + (ticks waiting / AGINGRATE), with a max of 99. If AGING is 0, it just returns the normal priority.



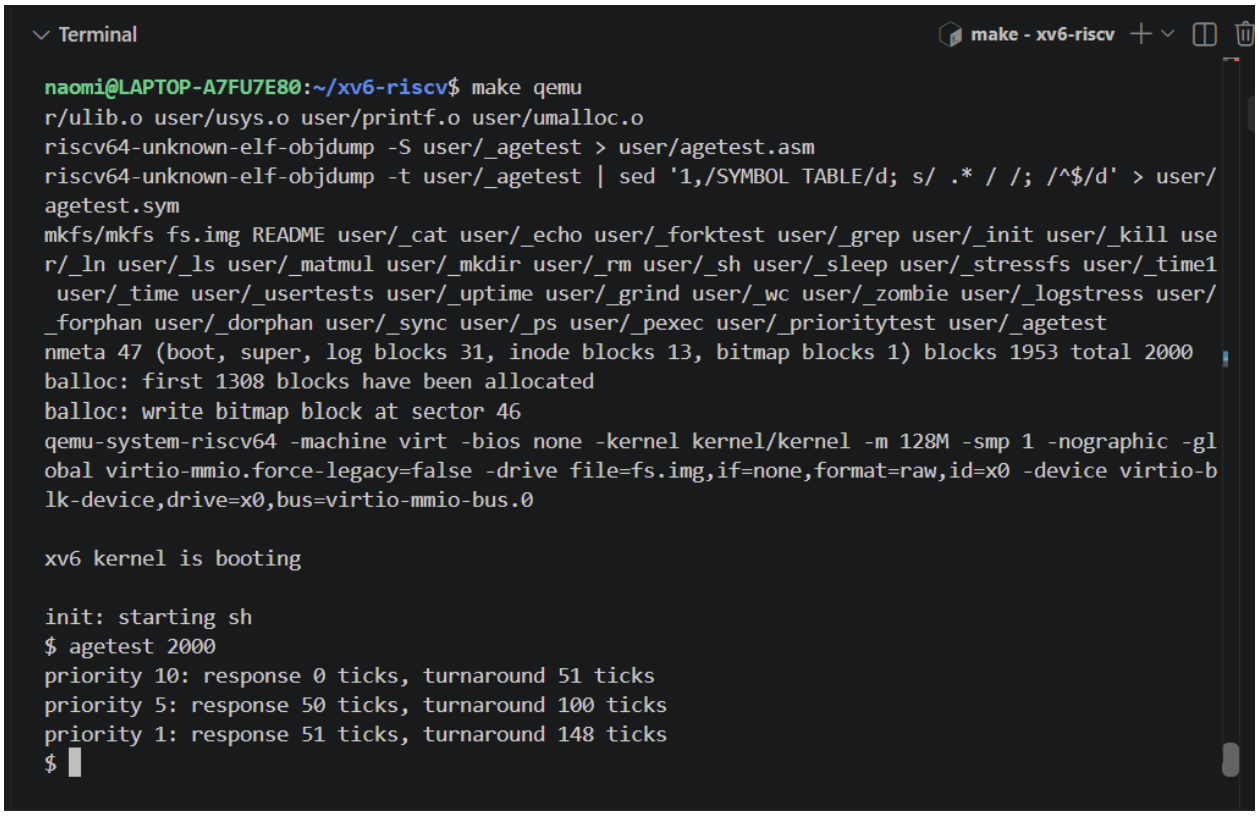
kernel/proc.c, scheduler(): the #if SCHEDULER == SCHED_PRIORITY part looks through the process table for the RUNNABLE process with the highest effective priority and runs it. The original round-robin code is still in the #else part. The search starts after the last process that ran, so if there is a tie the processes take turns.



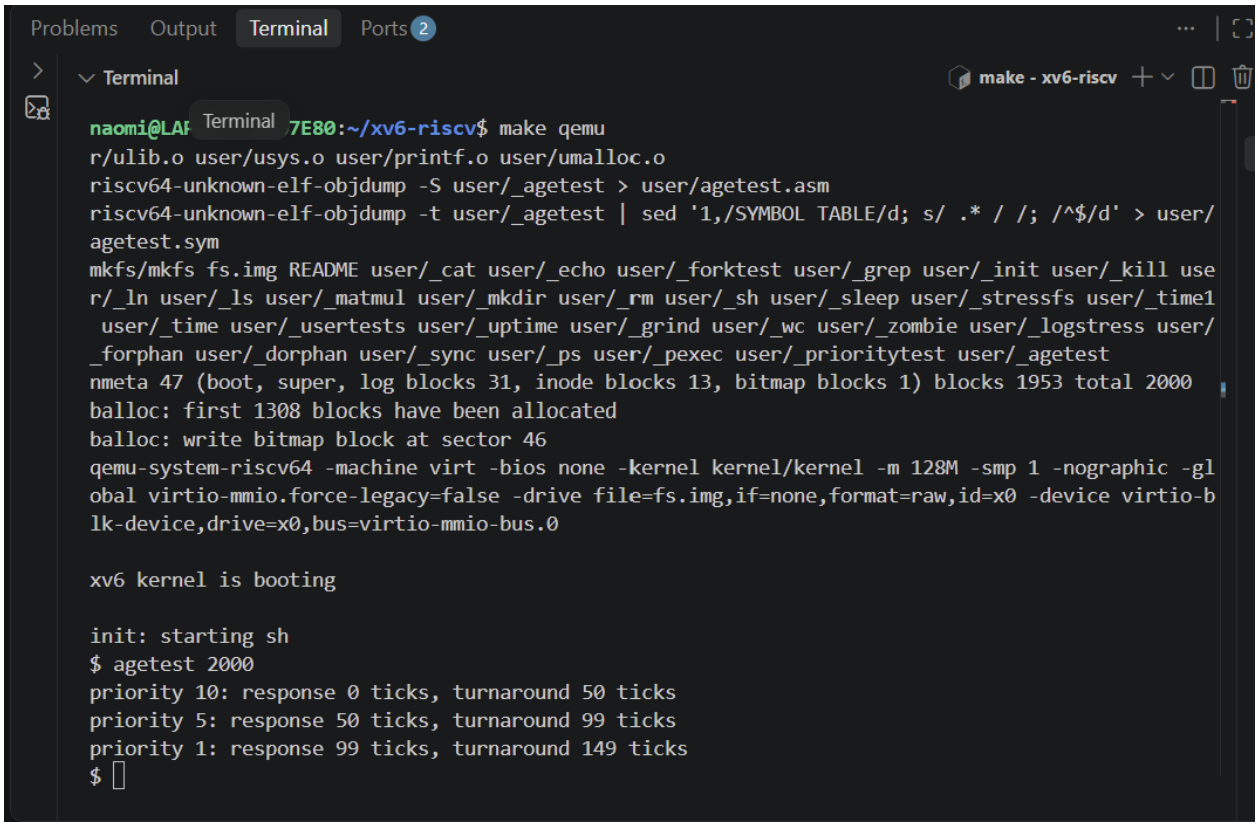
user/agetest.c: my test program for aging. The parent sets itself to priority 20 so it can start all 3 children before they run. Each child gets its priority (1, 5 or 10) by inheriting it from the parent at fork(). The child saves the time it first runs and the time it finishes, then prints its response time (first run - start) and turnaround time (finish - start). The argument (2000) controls how much work each child does.



Testing with aging ON (agetest 2000): agetest.c starts 3 processes with priorities 1, 5 and 10 and prints the response time (time until it first runs) and turnaround time (time until it finishes) of each one. Average response = (0 + 50 + 51) / 3 = 33.7 ticks. Average turnaround = (51 + 100 + 148) / 3 = 99.7 ticks.



Testing with aging OFF (AGING 0, same test): average response = (0 + 50 + 99) / 3 = 49.7 ticks. Average turnaround = (50 + 99 + 149) / 3 = 99.3 ticks. With aging, priority 1 got its first turn after 51 ticks instead of 99, because it aged up while it was waiting. The average response time went down about 32% and the average turnaround stayed about the same. So aging stops starvation without making the whole system slower.



Task 6. (5 pts) Summary

Summarize what you learned from doing this assignment. Describe any obstacles you encountered and if/how you overcame them.

In this assignment I learned how the xv6 scheduler picks which process runs next, and how to change it from round-robin to priority-based. I learned how to add new fields to struct proc and how to pass kernel info to a user program with a system call and either_copyout(). I also learned that a priority scheduler can make low priority processes starve, and that aging fixes it.

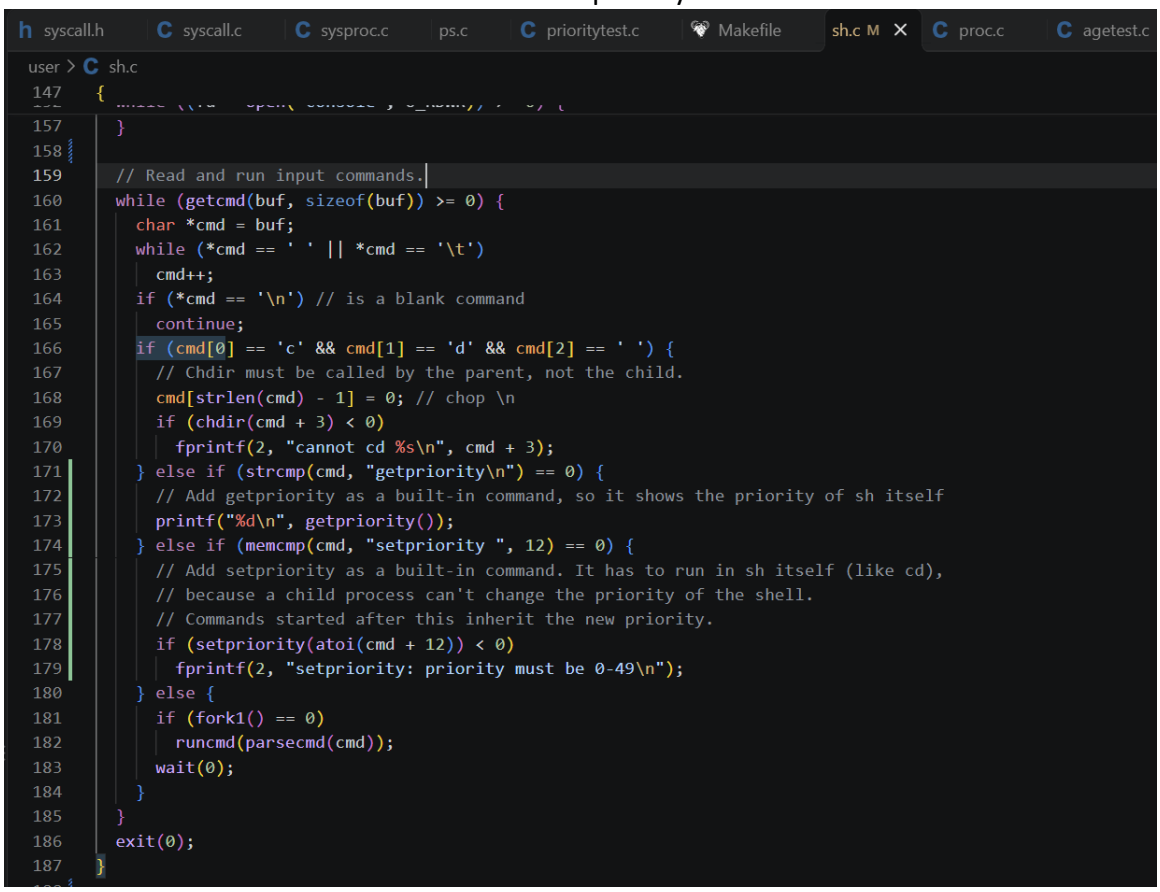
Obstacles: the provided ps.c didn't compile because of %l, so I changed it to %ld. copyout takes 5 arguments in my version of xv6, so I used either_copyout() instead. A few times I typed a new line over an existing line (like the safestrcpy line in getprocs), and the names in ps came out as garbage or blank. I fixed it by checking the function line by line. I also had to move enum procstate to pstat.h so ps.c could use it.

Extra Credit Task (10 points).

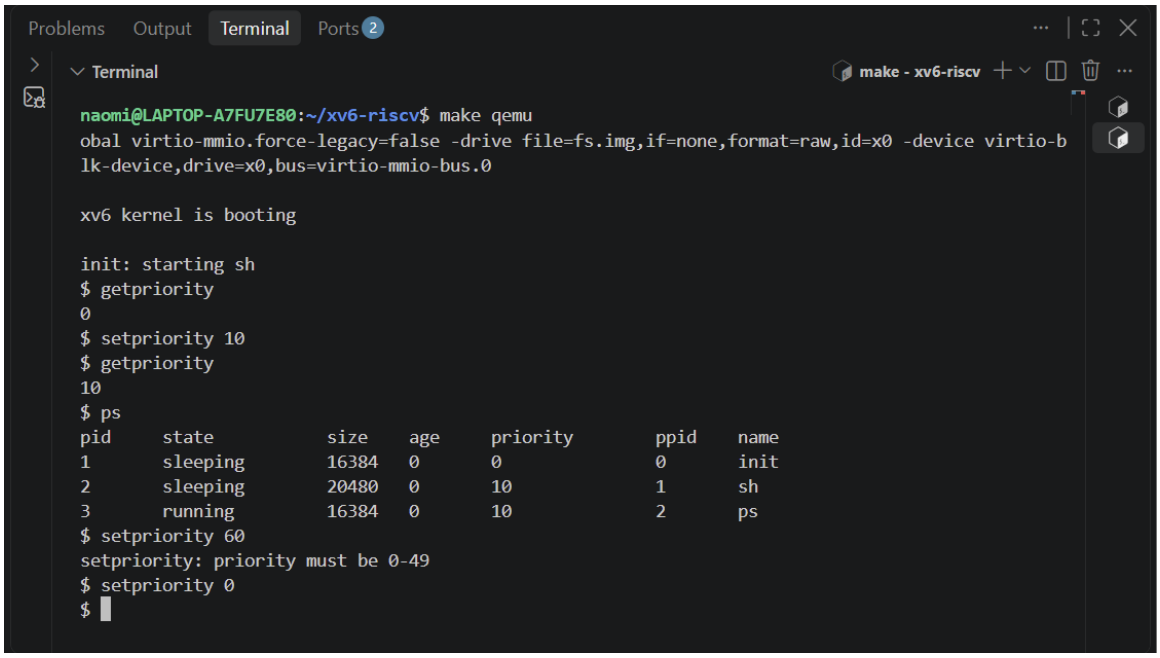
Explain how you implemented the getpriority and setpriority commands. Show how using these commands can improve the response time for interactive jobs.

I added getpriority and setpriority as built-in commands in the shell (user/sh.c), the same way

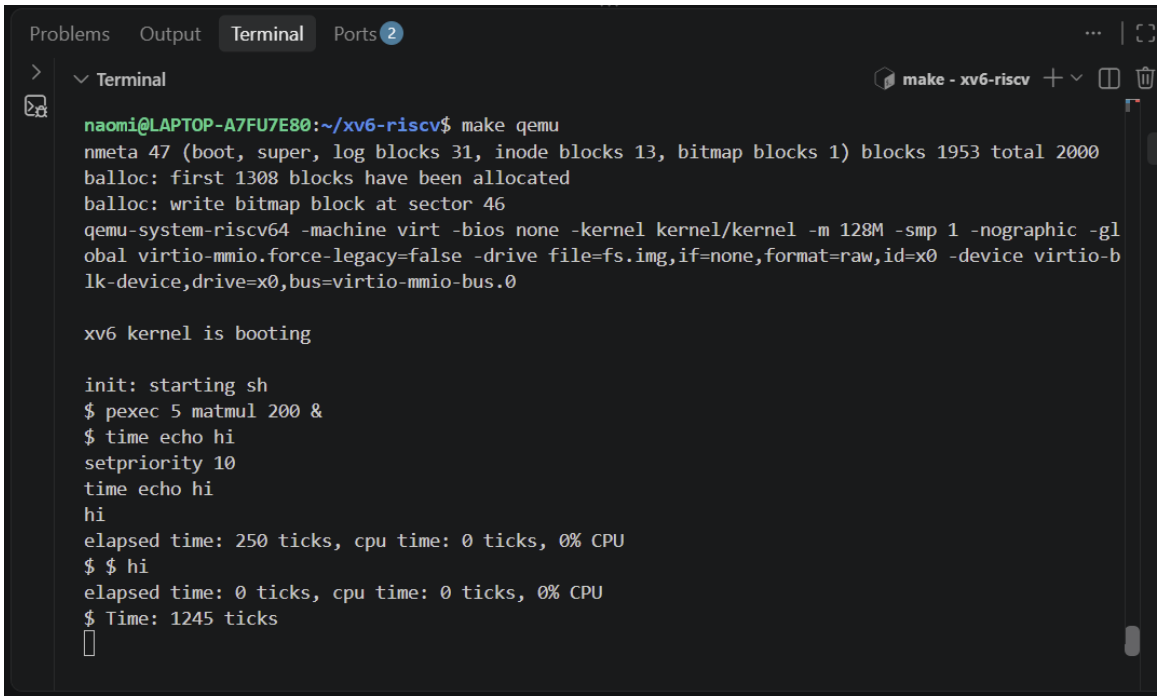
cd is built in. If they were normal programs, sh would fork a child to run them, and setpriority would only change the priority of that child, which exits right away. So sh checks the command line itself in main(): if it is getpriority, it prints the priority of sh with getpriority(). If it starts with "setpriority ", it calls setpriority() with the number after it (atoi), and prints an error if the number is not 0 to 49. Since sh doesn't fork for these, the priority of sh itself changes, and every command sh runs after that inherits the new priority.



Test A: getpriority shows 0. After setpriority 10, getpriority shows 10, and in ps both sh (pid 2) and the ps it started (pid 3) have priority 10, so the commands inherit the shell's priority. setpriority 60 is rejected, and setpriority 0 puts it back to 0.



Test B, response time for interactive jobs: I ran pexec 5 matmul 200 & in the background and then time echo hi with the shell at priority 0. sh and echo had a lower priority than matmul, so they had to wait until aging raised their effective priority above 5, and echo took 250 ticks. The shell was so slow that the next commands I typed showed up on the screen before it answered. Then I ran setpriority 10 and time echo hi again while matmul was still running. Now sh and echo had a higher priority than matmul, so echo ran right away: 0 ticks. matmul still finished afterwards (Time: 1245 ticks). This shows that giving the shell a higher priority makes interactive jobs respond much faster, while long CPU jobs keep running in the background at a lower priority.


Problems
Output
Terminal
Ports 2
Terminal
make - xv6-riscv
naomi@LAPTOP-A7FU7E80:~/xv6-riscv$ make qemu
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
balloc: first 1308 blocks have been allocated
balloc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -nographic -gl
obal virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-b
lk-device,drive=x0,bus=virtio-mmio-bus.0
xv6 kernel is booting
init: starting sh
$ pexec 5 matmul 200 &
$ time echo hi
setpriority 10
time echo hi
hi
elapsed time: 250 ticks, cpu time: 0 ticks, 0% CPU
$ $ hi
elapsed time: 0 ticks, cpu time: 0 ticks, 0% CPU
$ Time: 1245 ticks